Learn Python

Teacher Edition

Minute-by-minute session playbooks · transition scripts · predicted misconceptions · Socratic prompts

A 10-session, ~20-hour (two hours per session, + capstone) accelerated Python course for a PhD-in-Education learner with no prior coding experience.

Generated 2026-07-06 · Instructional content original

Learn Python — TEACHER Edition

Everything in the student syllabus, **plus** the instructor scaffolding: a minute-by-minute clock for each two-hour session, transition scripts (how to move between blocks without dead air), predicted misconceptions for *this* learner, Socratic prompts (DeepTutor-style — ask, don't tell), and an explicit **"if you're behind, cut this"** line per session.

Each two-hour session has two halves around a break: the **core** of the topic first, then a **Going deeper** block — genuinely new material (not stretched practice), with its own live demos and its own practice round. The 10 sessions run in five natural pairs (types→traps, control flow→data structures, functions→recursion, exceptions→files, regex→modules/OOP). Each pair shares a through-line; **name it out loud** when you open the second session of a pair, so consecutive sessions feel like one arc.

How to read this document

Each session has:

- **Covers** — core topic · going-deeper topics.
- **Pre-flight** — what to have open/ready before the student arrives.
- **The clock** — a 120-minute breakdown. Keep a timer visible. The numbers are targets, not law.
- **Transitions** — short scripted lines to hand off cleanly between blocks.
- **Predicted misconceptions** — where THIS learner (expert in research, novice in code) will stumble.
- **Socratic prompts** — questions to ask instead of explaining; let him derive it.
- **Cut line** — the first thing to drop if you're running over.
- **Homework** — what to assign at the close (specs live in `examples/session-NN/practice.md` → *Homework*).

The standard clock (adapt per session below)

- **0:00–0:05 — Warm-up.** Previous homework debrief + misconceptions log. Don't re-teach — quiz.
- **0:05–0:30 — Core concept.**
- **0:30–0:42 — Live demo I** (`demo.py`, predict-then-run; he types along).
- **0:42–1:00 — Practice I** (`practice.md` → *In class*).
- **1:00–1:08 — Break.** Honor it.
- **1:08–1:30 — Going deeper concept** (the deck's *Going deeper* slides).
- **1:30–1:40 — Live demo II** (the demo's *GOING DEEPER* sections).
- **1:40–1:53 — Practice II** (*In class — going deeper*).
- **1:53–2:00 — Recap + quiz + homework.**

Universal pacing principles (this learner is fast)

- **Talk less than you want to.** He reads fast and abstracts well. Default to "here's the rule, here's the trap, now you try."
- **Protect both practice blocks** (~30 min total, plus the live coding he types along with). If concept overruns, steal from your own talking, never from his typing.
- **The second hour is new material, not overflow.** If Practice I runs long, cut tasks, don't eat the Going-deeper block.
- **Predict-then-run is the engine, especially the Session 2 traps.** Always have him *commit to an answer out loud* before running.
- **The warm-up is homework-powered.** If homework wasn't done, do H1 together as the warm-up and move on.
- **Don't pad.** If a block ends early, bank the minutes for practice.
- **Carry a running "misconceptions log"** (DeepTutor "learning memory"): note every trap he hit; re-surface it as the next warm-up.

SESSION 1 — Running Python, Variables & Types

Covers: REPL vs script, variables as labels, the five core types, `input()` / `print()`, f-strings, casting, tracebacks · **deeper:** the full operator set (`// % **`), string methods, aligned f-strings, `import math`, `help()` / `dir()`, naming. **Pre-flight:** terminal + VS Code open; `examples/session-01/` ready; a deliberately broken line staged for the traceback.

The clock (120 min)

- **0:00–0:05 — Orientation.** Why Python, why this re-ordered path, how a session runs (core → break → deeper), what homework is for.
- **0:05–0:30 — Core concept.** REPL vs script. Variables as *labels on objects* (Connection Map #1). The five core types; `type(x)`. `input()` → always `str`. f-strings. Casting; `int(3.9)` truncates.
- **0:30–0:42 — Live I.** Build `greet.py`, then "years to graduation"; trigger and *read* one traceback (last line first).
- **0:42–1:00 — Practice I.** GPA reporter, age bucket, the string-concatenation trap.
- **1:00–1:08 — Break.**
- **1:08–1:30 — Deeper concept.** Arithmetic fully (`/ // % **`, precedence, `+=`); `%`-as-remainder uses; string methods (chaining, immutability, repetition); alignment f-strings; `print(sep=, end=)` and escape sequences; `import math` ("borrowing a toolbox — full story in S8"); `help()` / `dir()`; naming conventions & constants.
- **1:30–1:40 — Live II.** The demo's deeper sections: minutes→h/min, `% 3` groups, `.strip().title()` chain, the aligned report.
- **1:40–1:53 — Practice II.** Aligned gradebook lines; name normalizer; whole-units-and-remainder.
- **1:53–2:00 — Recap + quiz + homework.** `input()` → string; `int()` truncates; methods return new strings. Quiz S1.

Transitions

- Concept→live: *"Enough theory — watch me make these mistakes so you don't have to."*
- Break→deeper: *"You can run code. Now let's make the code worth running — the operators and string tools you'll touch every single day."*
- Close: *"You know the five types. Next session is the whole reason this course exists: the ways Python lets those types surprise you."*

Predicted misconceptions

- Expects `input()` to give a number → `"5" + "3" == "53"`.
- Thinks `=` asserts equality (math habit). Label-on-object now; pays off in S2 and S4.
- Expects `int(3.9)` to round; expects `.upper()` to change the string in place.

Socratic prompts

- "What type do you think `input()` hands back? How could we check?"
- "130 minutes. How do you get 2 and 10 out of it with two operators?"

- "You cleaned the name but nothing changed. What did `.strip()` actually hand you?"

Cut line: drop `help()` / `dir()` and the naming slide (point at the cheat sheet); never cut the traceback reading or either practice block. **Homework:** unit converter · traceback drill · type-prediction table.

SESSION 2 — The Dynamic-Typing Traps *(the most important session)*

Covers: `==` vs `is`, `bool ⊂ int`, float precision, cross-type comparison, truthiness, `isinstance` vs `type` · **deeper:** `None` semantics, the conversion matrix, `nan` / `inf`, mutable vs immutable + `id()`, `Decimal`. **Pre-flight:** `examples/session-02/`; the trap gauntlet open; `cheatsheets/traps-and-gotchas.md` ready to hand over.

The clock (120 min)

- **0:00–0:05 — Warm-up.** S1 homework: one traceback from his drill, one type-table row.
- **0:05–0:25 — Core concept.** This is the heart of the course; tell him so. `==` value vs `is` identity (Connection Map #3), `is None`; `bool ⊂ int` (Connection Map #4); floats are binary → `math.isclose` (Connection Map #5); `5 == "5"` vs `5 > "5"`; `isinstance` vs `type`; truthiness.
- **0:25–0:45 — The gauntlet (live + practice fused).** ~18 one-line traps, his prediction *out loud* before every run.
- **0:45–1:00 — Practice I.** The paper gauntlet, then start `clean_score()`.
- **1:00–1:08 — Break.**
- **1:08–1:30 — Deeper concept.** `None` properly (three kinds of "nothing"); the conversion matrix ("convert at the edge, once"); `nan` / `inf` and `math.isnan`; mutable vs immutable table + `id()` (*this explains every `is` surprise you just saw*); `Decimal` for exact arithmetic — and why you build it from strings.
- **1:30–1:40 — Live II.** Demo deeper sections: conversion matrix, `nan == nan`, `id()` on alias vs equal, `Decimal("0.1") + Decimal("0.2")`.
- **1:40–1:53 — Practice II.** Conversion predictions; the `Decimal` re-run; `describe(x)` (None vs `""` vs `0` vs `False` — his first real use of `isinstance(bool)` ordering).
- **1:53–2:00 — Recap + quiz + homework.** Hand over the trap cheat sheet as his permanent reference. Quiz S2.

Transitions

- Warm-up→concept: *"You know the five types. Here's the catch: Python lets them mix in ways that surprise everyone."*
- Break→deeper: *"You've seen WHAT surprises. Now the machinery underneath — what None really is, what `id()` shows, and the tool for when floats aren't good enough."*
- Close: *"Every trap you predicted wrong goes in your journal tonight — that's the homework, and it becomes your personal cheat sheet."*

Predicted misconceptions

- Assumes `is` is a stylistic `==` — nail it with the mutable-list demo, then `id()`.
- Trusts `==` on floats ("I round anyway") — the *cause* is binary storage, not data.
- Will write `Decimal(0.1)` and wonder why nothing improved — strings, not floats.
- `describe(False)`: forgets `False == 0` and reports "zero" — that's the planned stumble.

Socratic prompts

- "Two students, same 3.7 GPA. `==`? `is`? Why?"
- "`0.1 + 0.2` isn't `0.3` — is the bug in the data or in the computer? And when is `isclose` not good enough?" (→ money/bookkeeping → `Decimal`)
- "What's the difference between a blank cell, a zero score, and a missing student?"

Cut line: drop `inf`, `Fraction`, and the small-int cache wrinkle; **never** cut `==` / `is`, floats, `isinstance`, the gauntlet, or `describe(x)`. **Homework:** trap journal · `approx_equal` · `is_missing`.

SESSION 3 — Control Flow: Conditionals & Loops

Covers: `if/elif/else`, chained comparisons, `and` / `or` / `not`, `for` / `while`, `range`, `break` / `continue`, `enumerate` / `zip`, the validation loop · **deeper:** the ternary, `match/case`, `for/else`, nested loops, the named loop patterns. **Pre-flight:** `examples/session-03/`; a messy nested-`if` staged for refactor; `Ctrl+C` ready for the infinite loop.

The clock (120 min)

- **0:00–0:05 — Warm-up.** S2 homework: two trap-journal entries; quiz `approx_equal(0.1+0.2, 0.3)`.
- **0:05–0:30 — Core concept.** `if/elif/else` — and `if` vs `elif` (stacked `if`s are independent questions that ALL fire; show the double-label bug); **chained comparisons**; `and/or/not`, short-circuit, operand-return; `while` (+ infinite-loop demo); `for ... in`; `range` off-by-one; `break` / `continue`; the `while True:` validation pattern; `enumerate` / `zip` vs `range(len(...))`.
- **0:30–0:42 — Live I.** Likert→label classifier; roster average two ways; the "ask until valid" loop.
- **0:42–1:00 — Practice I.** Grade-band classifier (**test every boundary**: 89.999/90/90.001), logic drill, `zip` pass/fail, validation loop, the mutate-while-iterating trap.
- **1:00–1:08 — Break.**
- **1:08–1:30 — Deeper concept.** The ternary (tiny choices only); the boolean-return idiom (`return score >= 60` — hand the comparison straight back, use it bare in `if`); `match/case` (same-value-vs-literals ladders); `for/else` (search without a flag); nested loops (`break` exits inner only → function-and-`return` escape); the named patterns: accumulator, counter, best-so-far, sentinel.
- **1:30–1:40 — Live II.** Demo deeper sections: `match/case` classifier, `for/else` first-failing, nested sections×students, best-so-far.
- **1:40–1:53 — Practice II.** `match/case` rewrite (then: which reads better, and why?); `for/else` search; best-and-worst in one pass; the return-the-test rewrite.
- **1:53–2:00 — Recap + quiz + homework.** `if x == True` → `if x`; `range` excludes stop; name your pattern before you type. Quiz S3.

Transitions

- Concept→live: *"Watch me turn an ugly five-level `if` into three readable lines."*
- Break→deeper: *"You can branch and repeat. Now the idioms that make control flow READ well — and the four skeletons behind almost every loop you'll ever write."*
- Close: *"Next session: the containers worth looping over — and a list of dicts is secretly your dataset."*

Predicted misconceptions

- Writes `if score >= 90 and score < 100` — chain it.

- Boundary errors at cutoffs — make him test 89.999/90/90.001.
- `range(len(x))` habit — push `enumerate` / `zip`.
- Will over-apply `match/case` everywhere for a week — name its niche (one value vs literals).

Socratic prompts

- "`x = 5 and 0` — what's `x`? Why isn't it `True` / `False`?"
- "When does the `else` on a `for` run? What bookkeeping does it delete?"
- "Which of the four patterns is your grade-average loop? Your validation loop?"
- "A 95 printed three labels. What question did each `if` actually ask?"

Cut line: drop nested loops and the ternary slide (they resurface naturally); keep `match/case`, `for/else`, and both practice blocks. **Homework:** attendance labeler · number-guessing game · leap-year checker.

SESSION 4 — Data Structures

Covers: `list` / `tuple` / `dict` / `set`, slicing, list-of-dicts-as-dataset, comprehensions, `sorted(key=…)`, aliasing · **deeper:** unpacking, dict power methods, `Counter` / `defaultdict`, full set algebra, multi-key sorting, `copy` vs `deepcopy`. **Pre-flight:** `examples/session-04/`; the "list of dicts = tidy dataset" diagram (Connection Map #6); the aliasing demo staged.

The clock (120 min)

- **0:00–0:05 — Warm-up.** S3 homework: `label(79.9)`? `is_leap(1900)`? Re-run one boundary he missed.
- **0:05–0:30 — Core concept.** The four containers; indexing & slicing; nesting → **a list of dicts is a dataset**; list/dict comprehensions; `sorted(key=lambda …)`.
- **0:30–0:42 — Live I.** Build the roster, sort by score, dedupe with a `set`, rewrite a loop as a comprehension — then the **aliasing** bug and its fix.
- **0:42–1:00 — Practice I.** Rank, `{name: score}` comprehension, group pass/fail, dedupe, reproduce-then-fix aliasing.
- **1:00–1:08 — Break.**
- **1:08–1:30 — Deeper concept.** Unpacking (`head, *rest`, zip-loops ARE unpacking); dict power methods (`.get/.setdefault/.pop`, `|` merge); `Counter` ("your tally loop, one line"); `defaultdict(list)` grouping; set algebra (`& | - ^`, subset, `dict.fromkeys` ordered dedupe); multi-key tuple sorts + stability; `copy` vs `deepcopy`.
- **1:30–1:40 — Live II.** Demo deeper sections: shallow-vs-deep copy on nested lists, ordered dedupe; plus Counter/defaultdict from the earlier sections if not yet run.
- **1:40–1:53 — Practice II.** `Counter` + `.most_common`; `defaultdict` grouping; the multi-key sort; ordered dedupe.
- **1:53–2:00 — Recap + quiz + homework.** `=` aliases; `.sort()` returns `None`; tuple keys sort on multiple fields; nested → `deepcopy`. Quiz S4.

Transitions

- Open (the pair's thread): *"You can loop over data. This session: the containers worth looping — and a list of dicts is just a tidy dataset."*
- Break→deeper: *"You can store and sort. Now the toolkit that deletes half your loops — Counter, defaultdict, and sorts with tie-breaks."*
- Close: *"Data's handled. Next session we stop repeating ourselves: functions."*

Predicted misconceptions

- **Aliasing** is the big one — `b = a` doesn't copy; tie to S1 "label on object" and S2 `id()`.
- Expects `xs.sort()` to return the list.
- Expects `set` to keep first-seen order — show `dict.fromkeys`.
- Multi-key sort: will try `reverse=True` when only ONE key should descend — show the `-score` trick.

Socratic prompts

- "`b = a; a.append(99)` — what's in `b`? Why?"
- "Your tally loop is six lines. What one import makes it one line?"
- "Sort by score descending, ties by name ascending — what must the key return?"

Cut line: drop `^` symmetric difference and `deepcopy` detail (homework's grid covers sharing); keep Counter, defaultdict, and multi-key sorting. **Homework:** gradebook dict drill · frequency counter · fix-the-grid.

SESSION 5 — Functions, Scope & Reusability

Covers: `def`, parameters (positional/keyword/default, `*args` / `**kwargs`), `return` vs `print`, LEGB, docstrings, type hints, the mutable-default bug · **deeper:** functions as values, `lambda`, closures, a first decorator, keyword-only args, doctests. **Pre-flight:** `examples/session-05/`; the mutable-default demo staged (the headline); S4 inline code ready to refactor.

The clock (120 min)

- **0:00–0:05 — Warm-up.** S4 homework: the grid bug — *why* did every row change? (Aliasing. It should be automatic by now.)
- **0:05–0:30 — Core concept.** `def`; parameters; `return` vs `print`; scope (LEGB), why not `global`; docstrings; type hints (not enforced).
- **0:30–0:42 — Live I.** Refactor S4 code into `class_average` / `letter_grade`; the **mutable-default bug** live → fix with `None`.
- **0:42–1:00 — Practice I.** Grade-functions library; reproduce-then-fix the mutable default; `summary(*scores)`.
- **1:00–1:08 — Break.**
- **1:08–1:30 — Deeper concept.** Functions are objects (assign, pass, dispatch dict) — *`key=` was never magic*; `lambda` honestly; closures (`make_curver` factory; the capture-by-variable trap); a first decorator (`@announce` — "you'll USE `@property`, `@cache`, `@pytest.mark` forever; now you know what `@` does"); keyword-only `*`; doctests.
- **1:30–1:40 — Live II.** Demo deeper sections: the stats dispatch dict, curver factory, doctest run.
- **1:40–1:53 — Practice II.** `apply_to_all`; `make_curver`; write `@announce`; add a doctest and run it.
- **1:53–2:00 — Recap + quiz + homework.** Mutable default → `None`; forgot `return` → `None`; a decorator wraps; doctests keep docs honest. Quiz S5.

Transitions

- Concept→live: *"This next bug has burned every Python programmer once. Watch the list keep growing."*
- Break→deeper: *"You can write functions. Second hour: functions AS VALUES — the idea behind `key=`, decorators, and half of modern Python."*
- Close: *"A function can call other functions. Next session, the mind-bender: it can call itself."*

Predicted misconceptions

- Won't believe the default list persists — show twice; defaults evaluate once, at `def`.
- Thinks printing is returning.
- Decorators look like magic — write `curve = announce(curve)` long-hand FIRST, then show `@` is only sugar.

Socratic prompts

- "When does `bag=[]` actually run?"

- "What exactly did `sorted` receive when you wrote `key=lambda s: s['score']` last session?"
- "`make_curver(5)` returned something. What is it? Where does the 5 live now?"

Cut line: drop keyword-only args and `nonlocal`; **never** cut the mutable-default demo or the decorator (S6's `@cache` and S10's `@property` depend on it). **Homework:** tiny stats library · `mean_ignoring_none` · scope prediction.

SESSION 6 — Recursion & Recursive Thinking

Covers: base + recursive case, the call stack, recursion vs iteration, nested data, `RecursionError` · **deeper:** memoization (`@cache`), binary search, tree data, the explicit-stack escape, the decision checklist. **Pre-flight:** `examples/session-06/`; nested dict staged for `deep_sum`; the `runaway` overflow queued.

The clock (120 min)

- **0:00–0:05 — Warm-up.** S5 homework: `mean_ignoring_none(None)`? The scope drill — what error, why?
- **0:05–0:30 — Core concept.** Base case + recursive case; trace `orderings(3)` as a stack; recursion vs iteration; the cost (frames, no tail-call optimization, limit ≈ 1000); the prereq-chain example.
- **0:30–0:42 — Live I.** `countdown` / `factorial`; `deep_sum` on nested data; a `RecursionError` read together.
- **0:42–1:00 — Practice I.** Recursive `total`, `reverse`, `flatten`, `depth`, the two trap-fixes.
- **1:00–1:08 — Break.**
- **1:08–1:30 — Deeper concept.** Memoization: naive `fib` re-solves subproblems → `@functools.cache` (a decorator — yesterday's lesson working today); divide & conquer: binary search halves the problem ($\log_2$ intuition: 1,000 → ~10 steps); tree-shaped data (org chart: the function mirrors the tree); the explicit-stack version of `flatten` (same logic, no frames); the checklist: flat→loop, nested→recursion, repeated→cache, deep→stack.
- **1:30–1:40 — Live II.** Demo deeper sections: binary search with indented printout, org-tree count, `flatten_iter`.
- **1:40–1:53 — Practice II.** Binary search with step counter; `count_people` + `deepest`; `flatten` without recursion.
- **1:53–2:00 — Recap + quiz + homework.** Reachable base case; `return` the call; pick by the shape of the data. Quiz S6.

Transitions

- Open (the pair's thread): *"A function can call other functions. The mind-bender: it can call itself — exactly the tool for data defined in terms of itself."*
- Break→deeper: *"You can write A recursion. Now: when it's brilliant (halving, trees, caching) and how to escape when it isn't (the explicit stack)."*
- Close: *"You handle clean data beautifully. Next session: what to do when the data fights back."*

Predicted misconceptions

- Forgets to `return` the recursive call → silent `None`.
- Thinks binary search works on unsorted data — ask him why sorted matters.
- Assumes `@cache` helps every recursion — only *repeated* subproblems (deep_sum gains nothing).

Socratic prompts

- "What's the smallest input where the answer is obvious? That's your base case."
- "fib(35) computes fib(5) how many times? What would 'remembering' buy?"
- "1,000 sorted names — how many looks to find one? Why?"

Cut line: drop `deepest` and the mutual-recursion aside; **never** cut `deep_sum`, `@cache`, or binary search. **Homework:** `sum_digits` · `deep_count` · loop-vs-recursion paragraph.

SESSION 7 — Exceptions & Defensive Code

Covers: `try` / `except` / `else` / `finally`, exception types, `raise`, EAFP vs LBYL, `assert`, a first pytest test · **deeper:** the exception hierarchy & handler order, custom exceptions with data, `raise ... from`, logging, parametrized tests. **Pre-flight:** `examples/session-07/`; the dirty list staged; `pytest` installed and verified.

The clock (120 min)

- **0:00–0:05 — Warm-up.** S6 homework: trace `sum_digits(4823)`; why did `deep_count` need the bool check? (S2 pays rent.)
- **0:05–0:30 — Core concept.** Errors vs exceptions; `try/except/else/finally` — and keep the `try` block minimal; common types on sight; `raise`; EAFP vs LBYL; `assert` (developer check, not validation).
- **0:30–0:42 — Live I.** Harden `safe_int()` / `clean_likert()`; a first `pytest.raises` test, run it green.
- **0:42–1:00 — Practice I.** Clean the raw list into values + a rejection log; add a `raise`; one pytest test.
- **1:00–1:08 — Break.**
- **1:08–1:30 — Deeper concept.** The exception family tree; handlers tried top-down → specific before broad; catching a tuple; custom exceptions that carry data (`.value`); `raise ... from` (domain error outside, real cause inside); `finally` → the cleanup mindset (`with` is this, packaged — S8 preview); `logging` vs `print` (output vs diary, levels); pytest round 2: parametrize, arrange-act-assert, test the edges.
- **1:30–1:40 — Live II.** Demo deeper sections: except-order, the logging rejection log.
- **1:40–1:53 — Practice II.** Fix the except-order bug; `SurveyError` with `raise ... from`; convert the log to `logging`; parametrize the tests.
- **1:53–2:00 — Recap + quiz + homework.** Specific before broad; never bare `except:`; log the diary, print the output. Quiz S7.

Transitions

- Open: *"Your real survey data WILL have 'N/A' in a numeric column. Today we write code that shrugs it off."*
- Break→deeper: *"You can catch an error. Now: catch the RIGHT error, keep the evidence, and leave a paper trail — that's the difference between a script and a tool."*
- Close: *"You can survive one bad value. Real data is a file full of them — next session we open a real CSV."*

Predicted misconceptions

- Writes bare `except:` or puts `except Exception` first — the unreachable-handler demo cures it.
- Confuses `raise` with `return`, `assert` with validation.
- Will log at `DEBUG` and wonder where the output went — show the level threshold.

Socratic prompts

- "'seven' instead of 7 — catch it before (`if`) or after (`try`)? Trade-offs?"
- "Your program rejected four values silently. Who finds out, and when?"
- "Why does the order of the two `except` clauses matter? What's tried first?"

Cut line: drop `raise ... from` and shrink logging to the two-line teaser; keep except-order, `SurveyError`, and parametrize. **Homework:** `ask_int` · three pytest cases (as one parametrized test if he's ahead) · error triage.

SESSION 8 — Files, Libraries & Research Data

Covers: `open` / `with`, file modes, CSV via `DictReader` / `DictWriter`, the researcher's stdlib, the pandas teaser · **deeper:** `pathlib`, encodings, `csv.reader` vs `DictReader`, JSON for nested data, `datetime`, seeded `random`. **Pre-flight:** `examples/session-08/` with `students.csv` + `survey.csv`; `pip` works; `pandas` importable.

The clock (120 min)

- **0:00–0:05 — Warm-up.** S7 homework: one error-triage item; `ask_int(" 42 ")` — why does EAFP win?
- **0:05–0:30 — Core concept.** `open` / `with`; modes (`"w"` **truncates!**); CSV as dicts (ties to S4); the researcher's stdlib (`statistics`, `pathlib` first taste); `pip install`.
- **0:30–0:42 — Live I.** Read `students.csv` → list of dicts, class mean, write a summary CSV.
- **0:42–1:00 — Practice I.** Per-item survey means skipping dirty values; write `survey_summary.csv`; mean by major.
- **1:00–1:08 — Break.**
- **1:08–1:30 — Deeper concept.** `pathlib` full tour (`/` joins, `glob` / `rglob`, `read_text`); encodings (`utf-8`, the Excel BOM → `utf-8-sig`, mojibake); `csv.reader` vs `DictReader`, `delimiter=";"`; JSON for nested data (`True` → `true`, `None` → `null`); `datetime` (`strptime` / `strftime`, date math); `random` with a **seed** (reproducible sampling — methods-section gold); scripts that take arguments (`sys.argv`, `sys.exit`, the `argparse` pointer) and the `requests` API glimpse; the extended pandas teaser.
- **1:30–1:40 — Live II.** Demo deeper sections: `csv.reader` rows, date math, the seeded sample run twice.
- **1:40–1:53 — Practice II.** pathlib inventory; days-between; seeded sample; CSV → JSON; the argv-guarded `report.py`.
- **1:53–2:00 — Recap + quiz + homework.** `"w"` destroys; CSV values are strings; seed your randomness; JSON is its own language. Quiz S8.

Transitions

- Open (the pair's thread): *"You can survive one bad value. Real data is a file full of them."*
- Break→deeper: *"You've done one CSV by hand. Now the ecosystem around it — paths, encodings, dates, JSON, and reproducible sampling. This is the session your future scripts live in."*
- Teaser framing: *"Everything you did by hand, pandas does in five lines — next course, not today. Now you know what it does underneath."*
- Close: *"Numbers are clean. Next session: the messiest data of all — text."*

Predicted misconceptions

- Opens with `"w"` to read; iterates a file twice.
- Forgets CSV values are strings (`"91" + 1` — S2 again).
- Believes seeding makes randomness "fake" — reframe: it makes the *method* reproducible.
- Expects JSON to accept `True` / `None` spellings.

Socratic prompts

- "Why does `with` matter even if the program crashes?"
- "Your co-author reruns the sampling script. What guarantees they get the same participants?"
- "Nested per-item stats — why does CSV fight you and JSON doesn't?"

Cut line: drop encodings and `csv.reader` (leave the slide as reference) and shrink the pandas teaser; keep pathlib, datetime, and the seeded sample. **Homework:** attendance-report pipeline · JSON round-trip · your own CSV.

SESSION 9 — Regular Expressions & Text Cleaning

Covers: raw strings, the survival tokens, `search` / `fullmatch` / `findall` / `sub`, capture groups, when NOT to use regex · **deeper:** flags, `re.compile`, `VERBOSE`, named/non-capturing groups, greedy vs lazy, `sub` with a function, regex + files. **Pre-flight:** `examples/session-09/`; messy text, emails, "Last, First" names staged; regex101.com open (flavor: Python).

The clock (120 min)

- **0:00–0:05 — Warm-up.** S8 homework: what did JSON do to `True`? What dirty value did his own CSV throw?
- **0:05–0:30 — Core concept.** Why regex for a researcher (Connection Map #9); raw strings; the survival tokens; `.` matches any char; the four functions; capture groups.
- **0:30–0:42 — Live I.** Validate an email; extract dept+number; collapse whitespace; count hashtags; one case where `.split()` wins.
- **0:42–1:00 — Practice I.** Email validator, extract codes, hashtag count, the "Last, First" flip, the judgment call.
- **1:00–1:08 — Break.**
- **1:08–1:30 — Deeper concept.** Flags (`IGNORECASE`, `MULTILINE`); `re.compile` (name your patterns); `re.VERBOSE` (patterns with comments — reviewable!); named + non-capturing groups; greedy vs lazy (`.+` vs `.+?`); `re.sub` with a **function** (the anonymizer — a closure, S5 pays off); regex + files (S8 bridge).
- **1:30–1:40 — Live II.** Demo deeper sections: flag counts, greedy-vs-lazy brackets, the anonymizer.
- **1:40–1:53 — Practice II.** VERBOSE email; the anonymizer; the two-format date harvest; the flag count.
- **1:53–2:00 — Recap + quiz + homework.** `r"..."` always; guard `None`; lazy for bracketed things; compile what you reuse. Quiz S9.

Transitions

- Concept→live: *"Four functions cover almost everything. Every pattern is a raw string, no exceptions."*
- Break→deeper: *"You can write a pattern. Now write patterns a colleague can REVIEW — named, commented, compiled — and do the one trick plain sub can't: computed replacements."*
- Close: *"You can clean any string. Final session of new material: organizing your code so it's reusable."*

Predicted misconceptions

- Forgets raw strings; calls `.group()` on `None`.
- Reaches for greedy `.+` inside brackets/quotes — show the runaway match.
- Expects regex to understand meaning — it matches *form*.

Socratic prompts

- "A theme-mention count: form match or meaning match? What can regex actually promise?"

- "Why did `\\[(.+)\\]` swallow both course codes? What does the `?` change?"
- "Same participant name must always map to the same ID. Where does that memory live?" (→ the closure)

Cut line: drop `MULTILINE` and non-capturing groups; keep VERBOSE, greedy-vs-lazy, and the anonymizer. **Homework:** pattern drill · messy-name cleanup · domain harvest.

SESSION 10 — Modules, OOP & the Pythonic Toolkit

Covers: modules & the `__main__` guard, a class with `__init__` / `self` / `__str__`, a validating `@property`, inheritance, generators, `map` / `filter`, walrus · **deeper:** `__repr__` / `__eq__` / `__lt__`, dataclasses with `default_factory`, `@classmethod`, composition vs inheritance, class vs instance attributes, generator pipelines, project layout. **Pre-flight:** `examples/session-10/`; `grades.py` ready to import; the `Student` class and generator-exhaustion demo staged.

The clock (120 min)

- **0:00–0:05 — Warm-up.** S9 homework: run one of his patterns against an *invalid* example — did `fullmatch` catch it?
- **0:05–0:30 — Core concept.** Modules & the `__main__` guard; the `Student` class (`__init__`, `self`, `__str__`, validating `@property` — Connection Map #10); inheritance with `super()`.
- **0:30–0:42 — Live I.** Build the validating `Student`; `ana.gpa = 5.0` raises; the toolkit: comprehension, `map` / `filter`, generator exhaustion, walrus.
- **0:42–1:00 — Practice I.** Import from `grades.py`; the validating `Student`; `GradStudent(super())`; toolkit drills.
- **1:00–1:08 — Break.**
- **1:08–1:30 — Deeper concept.** `__repr__` / `__eq__` / `__lt__` → `sorted()` just works (and `__add__` — operator overloading in one line); dataclasses round 2 (`default_factory` = the S5 rule in class form; `frozen=True`); `@classmethod` alternate constructors (`from_row`: file format at the edge, objects inside — S8 bridge); composition over inheritance; class vs instance attributes (the trap → the law); generator **pipelines** (lazy stages, constant memory); from script to project (folder layout, the `def main()` convention, one module per concern).
- **1:30–1:40 — Live II.** Demo deeper sections: sortable `Score`s, `default_factory`, `from_row`, the two-stage pipeline.
- **1:40–1:53 — Practice II.** Sortable `Student`; the dataclass `Course`; `from_row`; the pipeline.
- **1:53–2:00 — Recap + quiz + homework + course wrap.** Dunders make objects native; mutable data in `__init__` / `default_factory`, always; compose first. Quiz S10. Frame the capstone: *"Next time, you drive."*

Transitions

- Open (the pair's thread): *"You can clean any string. Last step: organize code so it's reusable — functions into modules, data-plus-rules into classes."*
- Break→deeper: *"You've built a class that defends itself. Now make your objects feel NATIVE — printable, comparable, sortable — and wire generators into a pipeline that could eat a million rows."*
- Close: *"That's the toolbox — every tool earned its place. The capstone is where you prove it's yours."*

Predicted misconceptions

- `self` looks magical; generators exhaust.
- Puts `courses: list = []` in a dataclass — the S5 bug reborn; `default_factory` is the cure.

- Inherits when he should compose — "is-a or has-a?" out loud.

Socratic prompts

- "What must `sorted` be able to ask two Students, for no `key=` to be needed?"
- "Where should `roster = []` live so two Cohorts don't share students? Why?"
- "When does any work happen in `in_range(to_ints(raw))`? What forces it?"

Cut line: drop `frozen=True`, `@staticmethod`, and the project-layout slide; keep dunders, `default_factory`, `from_row`, and the pipeline. **Homework:** `Student` with computed GPA · `Cohort` class · Pythonic rewrite.

SESSION 11 (Optional) — Capstone

Role shift: you stop teaching and start *coaching*. He drives; you ask questions and unblock. Plan ~2 hours.

- **0:00–0:15 — Brief & plan.** He restates the goal and sketches the steps aloud (pseudocode). You only check the plan is sound.
- **0:15–1:05 — Build.** He codes the Gradebook & Survey Analyzer (`assessments/capstone-project.md`). Intervene only when stuck >3 min; prefer a question over an answer.
- **1:05–1:13 — Break.**
- **1:13–1:45 — Build, continued.** Finish the report CSV, then one stretch goal (a `Student` class, a regex validation, or the recursive nested-data total).
- **1:45–1:55 — Review.** Walk his code for the course's traps (identity, aliasing, mutable defaults, bare `except:`). Praise readability.
- **1:55–2:00 — Debrief & next steps.** Point to pandas/visualization as the genuine next course.

Coaching prompts: "What's your data structure?" · "What happens if that cell is blank?" · "Is that comparing value or identity?" · "Could that be one comprehension?"

Instructor's running checklist (use across all sessions)

- [ ] Timer visible; both practice blocks got their minutes; the break was honored.
- [ ] The Going-deeper block taught NEW material — it never became overflow practice time.
- [ ] Warm-up pulled from the homework and the misconceptions log — not improvised.
- [ ] Pair through-lines named out loud (S2, S4, S6, S8, S10 openings).
- [ ] Every trap demoed via **predict-then-run**, not narration.
- [ ] Each new concept hooked to the **Connection Map** before syntax.
- [ ] Student typed everything himself; you talked less than half the session.
- [ ] End-of-session quiz given; homework assigned by name; capstone kept in view as the destination.

Appendix A — Master Course Outline

Master Course Outline — Learn Python

> Single source of truth. The student and teacher syllabi both derive from this. **10 core two-hour sessions** + 1 optional capstone session (~20 hours core, ~22 with the capstone). Every session pairs a **core** hour with a **Going deeper** hour of new material, and assigns **homework (~30–45 min)** that does *not* count toward class time.

How the topics are sequenced

A typical intro-Python course teaches roughly: functions/variables → conditionals → loops → exceptions → libraries → unit tests → file I/O → regular expressions → OOP → assorted "power-tools."

We **re-sequence** for a fast adult learner, and we front-load the dynamic-typing traps — they're the whole reason this course exists. The 10 sessions run in five natural pairs, each pair sharing a through-line (name it when you cross between them):

1. **Types (S1) → the type traps (S2).** You can't reason about the traps until you know the types — so they lead, back to back, on days one and two.
2. **Control flow (S3) → data structures (S4).** Loops are most useful over the containers worth looping; a list of dicts is just a dataset.
3. **Functions (S5) → recursion (S6).** Recursion is a function that calls itself, so it lands while function mechanics are fresh — and nested data (from S4) is the payoff.
4. **Exceptions (S7) → files & research data (S8).** Surviving one dirty value scales straight into cleaning a whole CSV of them.
5. **Regex (S9) → modules & OOP (S10).** The two "finishing" skills: clean any text, then organize code into modules and a small class.

The power-tools (comprehensions, generators, `*args`, type hints, the walrus) are folded into the sessions where they naturally belong, not saved for the end.

Design rules (from the e-learning pipeline)

- **Every session = two hours = one topic, twice as deep**: warm-up → core concept → live example → practice → **break** → going-deeper concept → live example → practice → recap + quiz. The second hour is **new material** (never overflow practice); the two practice blocks total ~30 min of protected hands-on time.
- Every session assigns **homework** (in `examples/session-NN/practice.md` → *Homework*): ~30–45 min outside class, solutions included, reviewed in the next session's warm-up. The practice file's **In class — going deeper** block belongs to the second hour.
- Every session has 3–5 learning objectives across its core and going-deeper halves; every objective is testable in that session's quiz.
- Every abstract idea ships with a runnable, education-flavored example.
- Difficulty rises monotonically; nothing is used before it's introduced (except clearly-flagged teasers).

Session 1 — Running Python, Variables & Types

Objectives

1. Run Python interactively (REPL) and as a `.py` script; read a traceback without panic (last line first).
2. Use the core types (`int`, `float`, `str`, `bool`, `None`); check them with `type()`.
3. Do I/O with `input()` / `print()`, f-strings, and `int()` / `float()` / `str()` casting — and remember `input()` is *always* a `str`.

Going deeper (second hour): the full operator set (`// % **`), string methods, aligned f-strings, `print(sep=, end=)` & escape sequences, `import math`, `help()` / `dir()`, naming conventions. **Trap focus:** `input()` returns text (`"5" + "3"` is `"53"`); `int(3.9)` truncates; `print(a, b)` vs `print(a + b)`. **Homework:** unit converter; traceback drill; type-prediction table.

Session 2 — The Dynamic-Typing Traps *(the core of the course)*

Objectives

1. Distinguish **value equality (`==`)** from **identity (`is`)**; keep `is` for `None`.
2. Predict `bool ⊂ int` (`True == 1`, summing flags), int/float mixing, and **float precision** (`0.1 + 0.2`); compare floats with `math.isclose`.
3. Handle cross-type comparison (`5 == "5"` is `False`, `5 > "5"` raises); check types with `isinstance()` vs `type()`; reason about truthiness.

Why it leads: front-loading the traps — right after the types they concern — means every later session can assume the fluency. **Going deeper (second hour):** `None` semantics, the conversion matrix, `nan` / `inf`, mutable vs immutable + `id()`, exact arithmetic with `Decimal`. **Trap focus:** the ~18-trap predict-then-run gauntlet *is* the session. **Homework:** trap journal (5 traps in your own words); `approx_equal`; `is_missing`.

Session 3 — Control Flow: Conditionals & Loops

Objectives

1. Write `if` / `elif` / `else` with comparison & logical operators and **chained comparisons**; use `and` / `or` / `not` correctly (short-circuit, operand-return); avoid `if x == True`.
2. Write `for` / `while` loops; control them with `break` / `continue`; mind the `range` off-by-one; run the `while True:` validation loop.
3. Iterate Pythonically with `enumerate` / `zip` instead of `range(len(...))`.

Going deeper (second hour): the ternary, the boolean-return idiom, `match/case`, `for/else`, nested loops, the named loop patterns (accumulator, counter, best-so-far, sentinel). **Trap focus:** `if x == True`; stacked `if`s that all fire where an `elif` ladder was meant; `range(1,5)` excludes 5; mutating a list while iterating it; `range(len(...))`. **Homework:** attendance labeler (boundary testing); number-guessing game; leap-year checker.

Session 4 — Data Structures

Objectives

1. Choose between `list` / `tuple` / `dict` / `set`; index, slice, and nest them — a **list of dicts is a dataset**.
2. Build list/dict **comprehensions**; sort with `sorted(key=…)`.
3. Reason about **mutability & aliasing** (copy vs reference, `[[0]*3]*3` shared rows).

Going deeper (second hour): unpacking, dict power methods, `Counter` / `defaultdict`, full set algebra, multi-key sorting, `copy` vs `deepcopy`. **Trap focus:** aliasing (`b = a` shares the list); `.sort()` returns `None`; `dict.get()` vs `KeyError`; the shared-row grid. **Homework:** gradebook dict drill; frequency counter; fix-the-grid.

Session 5 — Functions, Scope & Reusability

Objectives

1. Define functions with positional, keyword, default, `*args`, and `**kwargs` parameters; know `return` vs `print`.
2. Explain scope (LEGB) and dodge the `UnboundLocalError` / `global` trap.
3. Document with docstrings and **type hints** (and know hints aren't enforced).

Going deeper (second hour): functions as values, closures, a first decorator, keyword-only arguments, doctests. **Trap focus:** the **mutable default argument** bug; forgetting to `return`; assigning to a global inside a function. **Homework:** a tiny stats library; `mean_ignoring_none(*values)`; scope-prediction drill.

Session 6 — Recursion & Recursive Thinking

Objectives

1. Write a recursive function with a correct **base case** and **recursive case**; trace the **call stack**.
2. Convert between recursion and iteration; know recursion's cost (`RecursionError`, no tail-call optimization, limit ≈ 1000).
3. Apply recursion to **naturally nested data** (nested lists/dicts/JSON) where one loop is awkward.

Going deeper (second hour): memoization (`@functools.cache`), binary search, tree-shaped data, the explicit-stack escape from the recursion limit. **Trap focus:** an unreachable base case → stack overflow; forgetting to `return` the recursive call. **Homework:** `sum_digits`; `deep_count` on a nested gradebook; loop-vs-recursion paragraph.

Session 7 — Exceptions & Defensive Code

Objectives

1. Handle errors with `try` / `except` / `else` / `finally`; `raise` deliberately; validate messy human/research input the EAFP way.
2. Know the common exception types on sight (`ValueError`, `TypeError`, `KeyError`, `FileNotFoundError`, …).
3. Use `assert` for developer checks (never input validation) and write a first `pytest` test.

Going deeper (second hour): the exception hierarchy & handler order, custom exceptions that carry data, `raise ... from`, `logging`, parametrized tests. **Trap focus:** bare `except:`; swallowing errors; `assert` ≠ validation. **Homework:** `ask_int` (EAFP validation loop); three more pytest cases; error triage.

Session 8 — Files, Libraries & Research Data

Objectives

1. Read/write text with `open` / `with` and understand file modes (why `"w"` is dangerous).
2. Load and write **CSV** survey/gradebook data with `csv.DictReader` / `DictWriter`; touch `json`.
3. `import` the researcher's stdlib (`statistics`, `datetime`, `pathlib`), `pip install` a package, and meet `pandas` in a guided teaser.

Going deeper (second hour): `pathlib`, encodings, `csv.reader` vs `DictReader`, JSON for nested data, `datetime`, seeded `random`, scripts with `sys.argv` / `sys.exit` (+ the `argparse` pointer), a `requests` API glimpse, the extended pandas teaser. **Trap focus:** `"w"` silently overwrites; forgotten `newline=""`; reading a file twice; CSV values are strings. **Homework:** attendance-report pipeline; JSON round-trip; run the pipeline on your own CSV.

Session 9 — Regular Expressions & Text Cleaning

Objectives

1. Write patterns with raw strings and the survival tokens; know when *not* to use regex.
2. Use `re.search` / `fullmatch` / `findall` / `sub` to validate, extract (capture groups), and clean real research text.

Going deeper (second hour): flags, `re.compile`, `re.VERBOSE`, named groups, greedy vs lazy, `re.sub` with a function replacement, regex over files. **Trap focus:** forgetting `r"..."`; `.` matches any char; `re.search` returns `None` — guard before `.group()`. **Homework:** pattern drill (IDs, phones, dates); messy-name cleanup; domain harvest.

Session 10 — Modules, OOP & the Pythonic Toolkit

Objectives

1. Split code into modules and `import` them; understand the `if __name__ == "__main__":` guard.
2. Model a domain entity with a small **class** (`__init__`, `self`, `__str__`, a validating `@property`, brief inheritance with `super()`).
3. Apply the **Pythonic toolkit**: comprehensions, `map` / `filter`, generators/ `yield`, the walrus `:=`.

Going deeper (second hour): `__repr__` / `__eq__` / `__lt__` and `__add__`, dataclasses with `default_factory`, `@classmethod` constructors, composition vs inheritance, generator pipelines, project layout with `def main()`. **Trap focus:** `self` confusion; a generator exhausts after one pass; the shared class variable; over-using a class where a function/dict fits. **Homework:** `Student` with computed GPA; a `Cohort` class; Pythonic rewrite of three loops.

Session 11 (Optional) — Capstone Project *(integrative)*

Objective: independently build one small, end-to-end program on a real-ish education dataset. Default brief: **"Gradebook & Survey Analyzer"** — read a CSV of students + Likert responses, clean and validate it, compute summary statistics, flag at-risk students, and write a report CSV. Alternative briefs are listed in `assessments/capstone-project.md`. Plan ~2 hours.

Coverage check (every core topic lands somewhere)

Topic	Where it lives now
Running Python, variables & types	S1
The dynamic-typing traps (`==` / `is`, floats, `bool⊂int`, `isinstance`)	S2
Conditionals & loops	S3
Data structures (list/tuple/dict/set)	S4
Functions, scope & reuse	S5
Recursion	S6 (nested data ties back to S4)
Exceptions & unit tests	S7
Files & libraries (CSV, `statistics`, `pandas` teaser)	S8
Regular expressions	S9
Modules & OOP	S10
Power-tools (comprehensions, `*args`, type hints, generators, `map` / `filter`, walrus)	S4, S5, S10

Scaling to the time budget

- **~14–16 hours:** keep every core hour; trim each Going-deeper block to its two or three starred essentials (each session's *cut line* names them).
- **~20 hours:** run S1–S10 as written, two hours each (recommended).
- **~22 hours:** add the S11 capstone.

Appendix B — Connection Map (education-research bridges)

Connection Map — Python ⇄ Education-Research Background

> From the *personalized-syllabus* method: each new programming concept is bridged to something the student already knows from education research, and — crucially — **where the bridge breaks** is named, so the new concept isn't quietly collapsed into the familiar one. Use these as the "hooks" when introducing each topic; they cut learning time dramatically for an expert adult.

---

1. Variables & assignment *(S1)*

- **Maps onto:** named quantities in a stats model — `n`, `mean`, `alpha`.
- **Where it breaks:** in math, `x = x + 1` is a contradiction; in Python `=` is an *action* (rebind the name), not an assertion of equality. A variable is a **label stuck on an object**, not a box that holds a value. This single reframing prevents most aliasing confusion later.

2. Types & dynamic typing *(S1–S2)*

- **Maps onto:** measurement levels — nominal/ordinal/interval/ratio. Python's `str` / `bool` / `int` / `float` are loosely analogous (categorical vs counted vs continuous).
- **Where it breaks:** SPSS/R columns have *one declared type per column*; a Python variable can hold a string now and an int later. The discipline a column gives you for free, you must supply yourself. This is exactly why the comparison traps in S2 exist.

3. `==` vs `is` *(S2)*

- **Maps onto:** the difference between **two questionnaires with identical answers** (equal in value) and **the same physical respondent** (identical individual). Two students can have the same GPA (`==`) without being the same person (`is`).
- **Where it breaks:** the analogy is exact for objects, but Python adds an implementation wrinkle (small-integer caching) that has no analogue in research — flagged as a "do not rely on this."

4. Boolean as a subclass of int (`True == 1`) *(S2)*

- **Maps onto: dummy coding** — you already encode "treatment = 1, control = 0" and then *sum* the dummies to count cases. Python literally does this: `sum([True, False, True]) == 2`.

- **Where it breaks:** in your data the 1/0 is a convention you imposed; in Python it's baked into the language, so `True + True == 2` works even when you didn't intend arithmetic on a flag.

5. Float precision (`0.1 + 0.2 != 0.3`) *(S2)*

- **Maps onto: rounding/measurement error**. You already never test two measured scores for exact equality; you use tolerances and report to 2 decimals.
- **Where it breaks:** here the error isn't in the data — it's in how the *computer* stores decimals in binary. Same instinct (`math.isclose`, round for display), new cause.

6. Lists / dicts / DataFrames *(S4, S8)*

- **Maps onto:** a `list` of `dict`s is a **tidy dataset**: each dict is a *row/respondent*, each key is a *variable/column*. A `dict` alone is one record's variable→value map.
- **Where it breaks:** a spreadsheet enforces rectangularity; a list of dicts does not — rows can have missing or extra keys, which is the source of many bugs (and why we validate in S7).

7. Functions *(S5)*

- **Maps onto:** a **statistical formula or a coding scheme**: defined once, applied to many cases, same rule every time → reproducibility, the thing you care about most as a researcher.
- **Where it breaks:** functions can carry *hidden state* (mutable defaults, globals) so the "same input → same output" promise can silently fail. That trap (S5) is the whole reason to learn scope.

8. Exceptions & validation *(S7)*

- **Maps onto: data cleaning** — out-of-range Likert values, blank cells, "N/A" typed into a numeric field. You already have a mental model of dirty data; exceptions are how code reacts to it.
- **Where it breaks:** cleaning in SPSS is a one-time batch pass; in a program you decide *at runtime*, per value, whether to skip, fix, or stop — a more granular control than a recode syntax file.

9. Regular expressions *(S9)*

- **Maps onto: search-and-filter in a corpus / qualitative coding** — finding every response matching a pattern, extracting IDs, normalizing free-text.
- **Where it breaks:** regex matches *surface form*, not meaning. It's powerful for structure (emails, dates, IDs) and treacherous for semantics — the opposite trade-off from human coding.

10. Classes / OOP *(S10)*

- **Maps onto:** an **operational definition / construct** — a `Student` class bundles the attributes and behaviors you've decided "count" as a student, like an operationalized variable.

- **Where it breaks:** a construct is a measurement choice; a class is also *behavior* (methods) and *enforced rules* (validation in setters), so it's a construct that can defend its own integrity.

11. Recursion *(S6)*

- **Maps onto:** a **hierarchy / nested structure** — coding schemes with sub-codes, threaded discussion data, folder trees of data files, nested JSON survey exports. A procedure "defined in terms of itself" mirrors data that contains smaller copies of itself.
- **Where it breaks:** recursion is elegant only when the structure is genuinely nested and the base case is reachable; for a flat sequence, a loop is clearer, and Python's stack limit (~1000, no tail-call optimization) makes very deep recursion a liability rather than a virtue.

Questions to hold while learning (Socratic spine)

These recur across sessions; the teacher should keep returning to them.

1. *Is this a question about value, or about identity?* (drives `==` vs `is`, copy vs alias)
2. *What type is this right now, and who decided that?* (dynamic typing discipline)
3. *Could this input be dirty? What happens if it is?* (defensive mindset)
4. *Is the same rule guaranteed to run the same way every time?* (reproducibility / pure functions)
5. *Is there a more Pythonic, more readable way to say this?* (the Zen of Python's "readability counts")

Appendix C — Per-Session Quizzes & Answer Keys

Per-Session Quizzes (with answer keys)

Each quiz is 5–6 questions (the last one or two from the Going-deeper hour), ~8 minutes, given at the end of the session. Every question maps to a session objective. **Teacher:** ask the student to *predict* code output before they run it — the surprise is the assessment. Answers are at the end of each session block.

Session 1 — Running Python, Variables & Types

1. What type does `input("x: ")` return, always?
2. What does `"5" + "3"` evaluate to? How do you get `8`? And what is `int(3.9)`?
3. What does `f"{0.873:.1%}"` produce?
4. In a traceback, which line do you read first — and what two things does it tell you?
5. What are `17 // 5` and `17 % 5` — and name one practical use of `%`.

Answers: 1. `str`. 2. `"53"`; use `int("5") + int("3")`; `int(3.9)` is `3` (truncates, doesn't round). 3. `"87.3%"`. 4. The **last** line: the exception's name (what kind of problem) and the message (the offending value or detail). 5. `3` and `2`; `%` gives the remainder — even/odd tests, rotating IDs into k groups, splitting minutes into h + min.

Session 2 — The Dynamic-Typing Traps

1. `a = [1,2]; b = [1,2]` — is `a == b`? Is `a is b`? Why?
2. What is `True + True`? Why?
3. Is `0.1 + 0.2 == 0.3` True or False? How *should* you compare them?
4. What does `5 == "5"` give? What about `5 > "5"`?
5. Which of these are falsy: `"0"`, `""`, `[0]`, `[]`, `None`, `0.0`?
6. Why does `Decimal("0.1") + Decimal("0.2") == Decimal("0.3")` hold when the float version doesn't — and why build Decimals from strings?

Answers: 1. `==` True (same value), `is` False (different objects in memory).

2. `2` — bool is a subclass of int (`True` acts as `1`). 3. False (binary float rounding); use `math.isclose(0.1 + 0.2, 0.3)` or round. 4. `False` (no error); `5 > "5"` raises `TypeError` — Python can't *order* int vs str. 5. `""`, `[]`, `None`, `0.0` are falsy; `"0"` and `[0]` are truthy (non-empty). 6. `Decimal` stores base-10 digits exactly, so there's no binary rounding; built from a float (`Decimal(0.1)`) it would inherit the float's error.

Session 3 — Control Flow: Conditionals & Loops

1. Rewrite `if x >= 90 and x < 100:` using a chained comparison.
2. What is `5 and 0`? What is `0 or "hi"`? Why aren't they `True` / `False`?
3. What does `list(range(1, 5))` produce?
4. What goes wrong when you `remove` items from a list while looping over it — and what's the fix?
5. When does the `else` clause on a `for` loop run?
6. With `score = 95`, three stacked `if score >= 60 / 80 / 90:` prints run — what happens, and what one keyword fixes it?

Answers: 1. `if 90 <= x < 100:`. 2. `0` and `"hi"` — `and` / `or` return an *operand*, not a bool. 3. `[1, 2, 3, 4]` (5 excluded — off-by-one). 4. Removal shifts the remaining indices, so elements get skipped; build a new list instead (`[x for x in xs if keep(x)]`) or iterate over a copy. 5. Only when the loop finishes **without** `break` — the "searched everything, found nothing" branch. 6. All three labels print (each `if` is an independent question); chaining with `elif` makes one ladder where only the first hit runs.

Session 4 — Data Structures

1. Which are mutable: list, tuple, dict, set?
2. `a = [1,2]; b = a; a.append(3)` — what is `b`? Why?
3. (Practical) Write a one-line dict comprehension mapping each name in `names` to `0`.
4. `xs.sort()` vs `sorted(xs)` — what does each return?
5. You need "have I seen this ID before?" over thousands of IDs — which structure, and why?
6. What does `Counter(['a','b','a']).most_common(1)` return?

Answers: 1. list, dict, set are mutable; tuple is not. 2. `[1, 2, 3]` — `b` is an alias for the *same* list (assignment doesn't copy). 3. `{n: 0 for n in names}`.

4. `.sort()` sorts in place and returns `None`; `sorted()` returns a *new* sorted list.
5. A `set` — membership tests are fast and duplicates are impossible by construction. 6. `[('a', 2)]` — a list of (value, count) pairs, biggest first.

Session 5 — Functions, Scope & Reusability

1. What's the difference between `return` and `print`?
2. Why is `def f(x, items=[])` dangerous? What's the fix?
3. What does a function with no `return` statement return? Are type hints enforced at runtime?
4. `count = 0` at top level, then inside a function `count = count + 1` — what error, and why?
5. In one sentence: what does a decorator do, and what is `@d` above a `def f` sugar for?

Answers: 1. `return` hands a value to the caller; `print` only displays it.

2. The default list is created once, at `def` time, and persists across calls; use `items=None` then create inside. 3. `None`; and no — hints are documentation (`mypy` checks them optionally). 4. `UnboundLocalError` — the assignment makes `count` local to the function, so the right-hand side reads a local that doesn't exist yet. 5. It wraps a function with extra behavior; `@d` is exactly `f = d(f)`.

Session 6 — Recursion & Recursive Thinking

1. What two parts must every recursive function have?
2. What error comes from a base case that's never reached, and why doesn't Python just keep going?
3. Why is recursion a natural fit for nested data (a list of lists, or nested JSON)?
4. The recursive case computes `n * f(n - 1)` but has no `return` in front — what does a call produce?
5. Naive recursive `fib(35)` takes seconds but `@functools.cache` makes it instant — what does the cache change?

Answers: 1. A **base case** (stops) and a **recursive case** (calls itself on a smaller input, *toward* the base case). 2. `RecursionError` — Python has no tail-call optimization, so each pending call keeps a stack frame until the limit (~1000).

3. The data is *defined in terms of itself* (a list may contain lists), so a function defined in terms of itself mirrors its shape and reaches every level. 4. `None` — the value is computed and thrown away; the function falls off the end. 5. Each distinct input is computed once and remembered, so the exponential tree of repeated subcalls collapses into ~35 lookups.

Session 7 — Exceptions & Defensive Code

1. Which exception does `int("N/A")` raise? Wrap `int(value)` so it returns `None` on failure.
2. Why is a bare `except:` dangerous?
3. When should you use `raise` vs `assert`?
4. Name the two validation styles — and give the EAFP version of converting `value` to an int.
5. Why must `except ValueError:` come before `except Exception:`?

Answers: 1. `ValueError`; `try: return int(value)` / `except (ValueError, TypeError): return None`.

2. It catches *everything* (even Ctrl+C and your own typos) and can hide bugs.
3. `raise` to validate real/untrusted input; `assert` for developer sanity checks (it can be disabled with `python -O`). 4. LBYL ("look before you leap": `if value.isdigit():`) vs EAFP ("easier to ask forgiveness"): `try: n = int(value)` / `except ValueError: ...`. 5. Handlers are tried top-down and `Exception` matches a `ValueError` too — placed first it would swallow everything, leaving the specific handler unreachable.

Session 8 — Files, Libraries & Research Data

1. What does opening a file in `"w"` mode do to existing contents? Why prefer `with open(...)`?

2. After `csv.DictReader`, what type is each row?

3. CSV values read from a file are what type — and what must you do with numbers?
4. Your second loop over an open file object sees nothing — why, and what's the fix?
5. What do `strptime` and `strftime` each do — and why seed `random` in an analysis script?
6. Your script runs as `python3 report.py survey.csv` — what is `sys.argv`, and what sits at index 0?

Answers: 1. Truncates it to empty *immediately*; `with` auto-closes the file even if the code crashes. 2. A `dict` keyed by the header row. 3. Strings; convert with `int()` / `float()`. 4. The file cursor is exhausted after one pass — re-open the file (or read it into a list first). 5. `strptime` parses text → datetime; `strftime` formats datetime → text; seeding makes the "random" sample reproducible for your methods section. 6. A plain list of strings, `['report.py', 'survey.csv']`; index 0 is always the script's own name, so real arguments start at 1 — check `len(sys.argv)` and `sys.exit("usage...")` before trusting them.

Session 9 — Regular Expressions & Text Cleaning

1. In regex, what does `.` match? How do you match a literal dot?
2. Why write regex patterns as raw strings `r"..."`?
3. What does `re.search` return when there's no match, and what must you do before `.group()`?
4. `re.search` vs `re.fullmatch` — which one is for validation, and why?
5. What does `re.IGNORECASE` change, and what is `re.VERBOSE` for?

Answers: 1. Any character (except newline); use `\.` for a literal dot. 2. So backslashes aren't treated as Python string escapes. 3. It returns `None`; check `if m:` before `m.group()` or you'll hit an `AttributeError`. 4. `fullmatch` — it anchors both ends, so the *whole* string must fit the pattern; `search` would accept a valid-looking fragment inside garbage. 5. `IGNORECASE` matches regardless of case; `VERBOSE` lets a pattern span lines with comments so a colleague can actually review it.

Session 10 — Modules, OOP & the Pythonic Toolkit

1. In a class, what is `self`?
2. What happens if you iterate a generator twice?
3. Why doesn't a module's `if __name__ == "__main__":` block run when you `import` it?

4. `class Course: students = []` — what goes wrong when two `Course` objects enroll students, and what's the fix?

5. In a dataclass, why write `courses: list = field(default_factory=list)` instead of `= []`?

Answers: 1. The current instance ("this particular object"). 2. The second pass is empty — a generator is exhausted after one iteration. 3. On import, `__name__` is the module's name, not `"__main__"`, so the block is skipped. 4. `students` is a *class* variable shared by every instance, so both courses see one combined list; give each its own in `__init__`: `self.students = []`. 5. A bare `[]` default would be created once and shared by every instance — the Session 5 mutable-default bug in class form; `default_factory` builds a fresh list per instance.

Scoring guide (formative, not graded)

- **All correct, explained why:** ready for the next session (or the capstone).
- **Right answer, fuzzy why:** re-do that session's `traps-and-gotchas` rows.
- **Wrong on Session 2 items:** revisit the type traps before continuing — they're load-bearing for everything after.